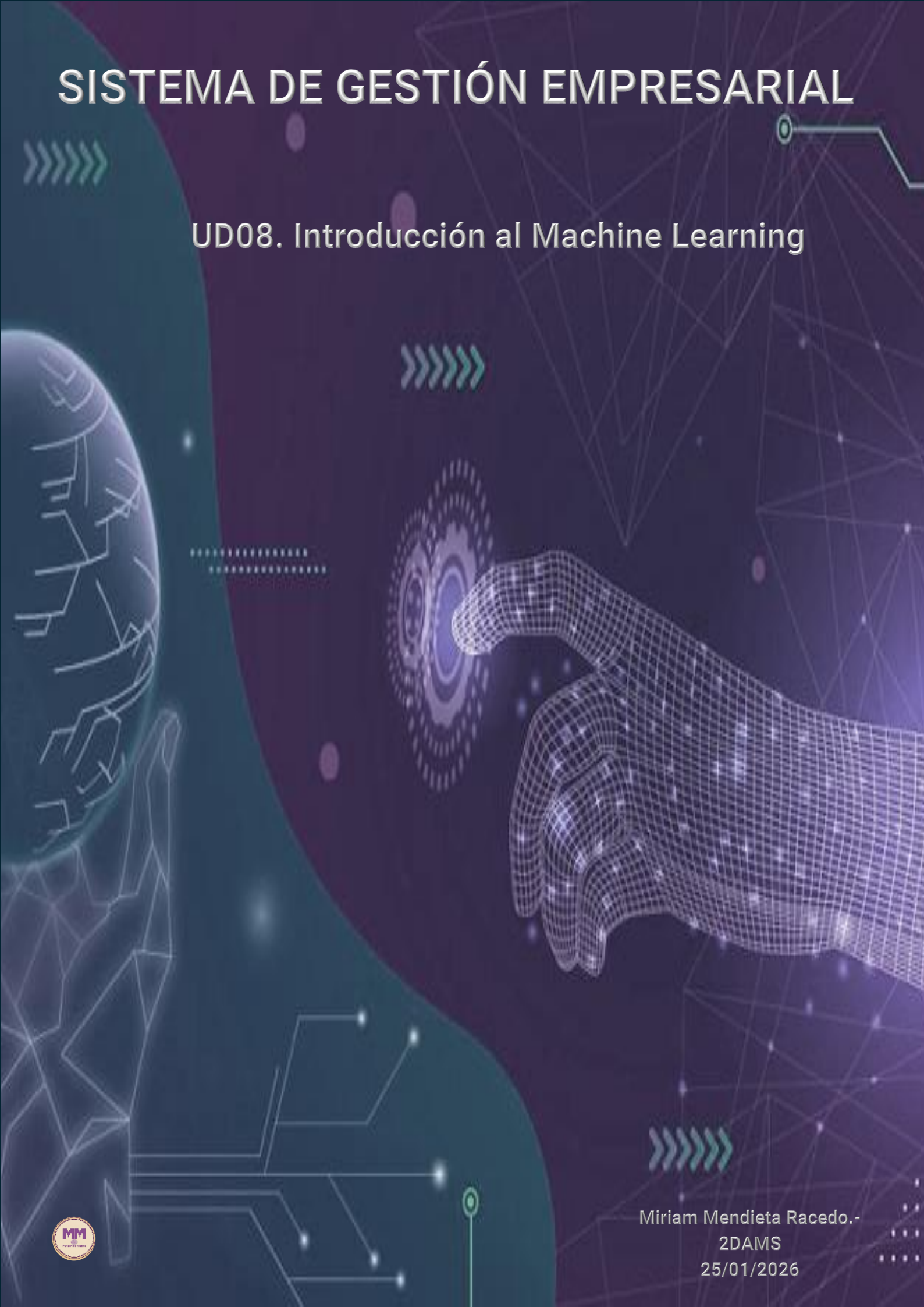
SISTEMA DE GESTIÓN EMPRESARIAL

UD08. Introducción al Machine Learning



Miriam Mendieta Racedo.-
2DAMS
25/01/2026

ÍNDICE

INTRODUCCIÓN **1**

ACTIVIDAD 01 3

MACHINE LEARNING **4**

PREPARACION DEL ENTORNO **6**

Descarga del Repositorio 6
Instalación de Dependencias 6
Configuración de Visual Studio Code 6
Jupyter (Juypter Notebooks) 7
Pandas 9
NumPy 9
Matplotlib 9
Seaborn 10
Scikit-learn 10

ANÁLISIS DE NOTEBOOKS **11**

Manipulación de Datos con Pandas 12
Regresión Lineal 14
k-Nearest Neighbor 16
Árbol de decisión 17

BIBLIOGRAFIA **19**

INTRODUCCIÓN

La implementación de modelos de Machine Learning en el entorno empresarial ha evolucionado de ser una simple opción para convertirse en una necesidad estratégica ineludible. Esta transformación tecnológica no solo moderniza la operativa de las empresas, sino que también empodera a los acatores responsables para tomar decisiones fundamentadas en datos concretos, abandonando enfoques tradicionales basados en suposiciones.

La Integración de Herramientas Avanzadas

La combinación de herramientas como Jupyter ,Pandas, Matplotlib, en el proceso de análisis de datos permite una gestión más eficaz de la información. Estas herramientas facilitan:

- Procesamiento Eficiente de Datos: La manipulación de grandes conjuntos de datos se vuelve más ágil, lo que reduce el tiempo de análisis y permite una respuesta más rápida a las necesidades del mercado.
- Visualización Clara: La capacidad de representar datos de manera gráfica ayuda a los equipos no técnicos a comprender mejor las conclusiones, lo que fomenta una cultura empresarial más colaborativa.

Metodologías Modernas en la Toma de Decisiones

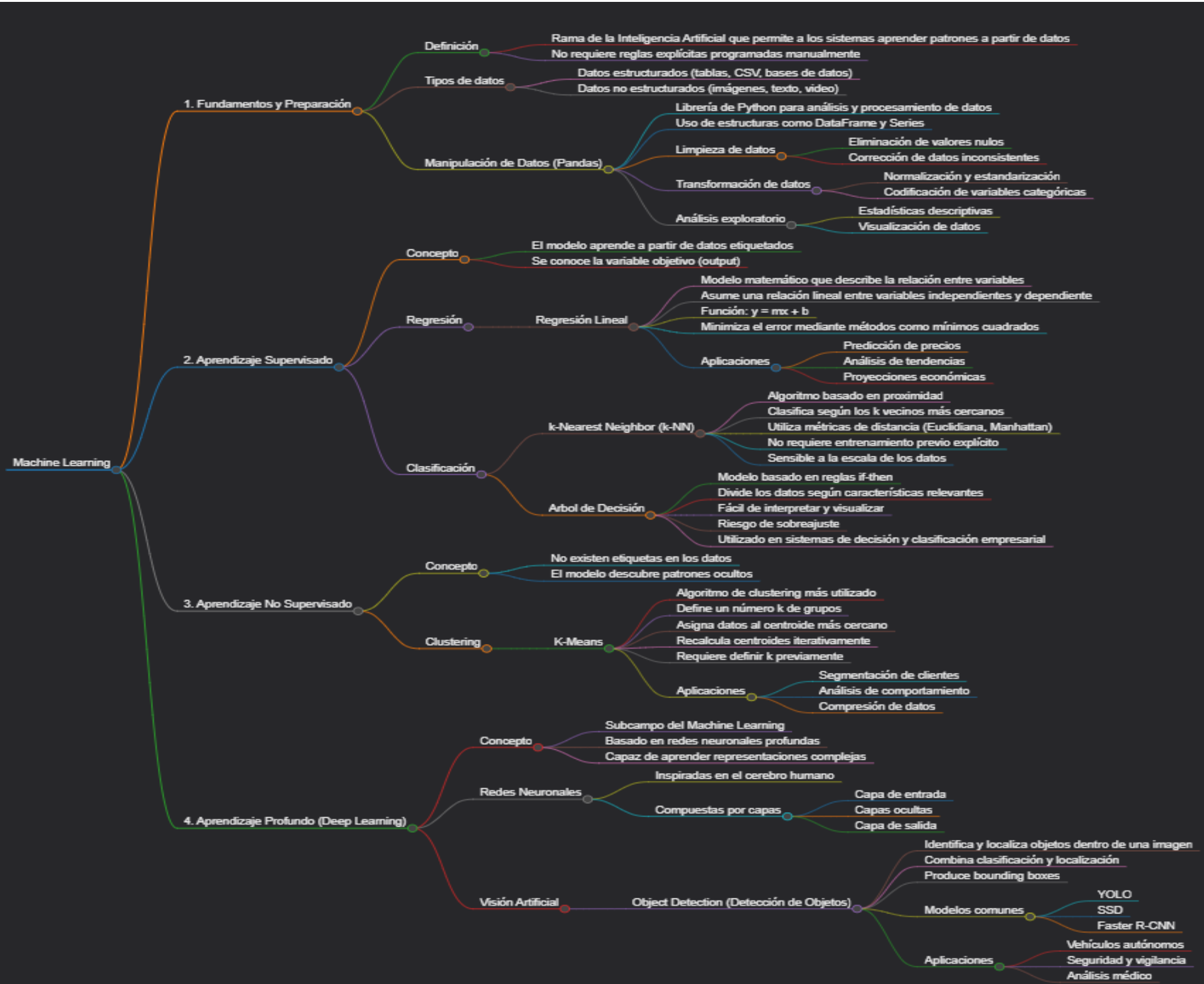
Implementar modelos de Machine Learning también implica adoptar metodologías ágiles que permiten un ciclo de mejora continua. Esto incluye:

- Prototipado Rápido: La creación de prototipos permite a las empresas experimentar con diferentes modelos y enfoques, adaptándose a los cambios en el entorno de negocio.
- Feedback Continuo: La colaboración entre departamentos, facilitada por herramientas interactivas, asegura que las decisiones se fundamenten en un análisis conjunto y multidimensional.

El Futuro del Machine Learning en los Negocios

La tendencia hacia la digitalización y la automatización continuará, y aquellas empresas que integren modelos de Machine Learning en sus respectivos sectores estarán mejor posicionadas para competir. En un entorno donde la adaptabilidad y la velocidad son clave, la capacidad de tomar decisiones basadas en datos no es solo ventajosa, sino esencial.

En conclusión, el Machine Learning no solo representa una herramienta poderosa para la toma de decisiones, sino que también redefine la forma en que las empresas operan y se adaptan al mercado. Esta evolución hacia una estrategia más basada en datos marca un cambio significativo hacia una gestión empresarial más eficaz, impulsando así el crecimiento y la innovación.


Machine Learning
1. Fundamentos y Preparación
Definición
Rama de la Inteligencia Artificial que permite a los sistemas aprender patrones a partir de datos
No requiere reglas explícitas programadas manualmente
Tipos de datos
Datos estructurados (tablas, CSV, bases de datos)
Datos no estructurados (imágenes, texto, video)
Manipulación de Datos (Pandas)
Librería de Python para análisis y procesamiento de datos
Uso de estructuras como DataFrame y Series
Limpieza de datos
Eliminación de valores nulos
Corrección de datos inconsistentes
Transformación de datos
Normalización y estandarización
Codificación de variables categóricas
Análisis exploratorio
Estadísticas descriptivas
Visualización de datos
2. Aprendizaje Supervisado
Concepto
El modelo aprende a partir de datos etiquetados
Se conoce la variable objetivo (output)
Regresión
Regresión Lineal
Modelo matemático que describe la relación entre variables
Asume una relación lineal entre variables independientes y dependiente
Función: y = mx + b
Minimiza el error mediante métodos como mínimos cuadrados
Aplicaciones
Predicción de precios
Análisis de tendencias
Proyecciones económicas
Clasificación
k-Nearest Neighbor (k-NN)
Algoritmo basado en proximidad
Clasifica según los k vecinos más cercanos
Utiliza métricas de distancia (Euclidiana, Manhattan)
No requiere entrenamiento previo explícito
Sensible a la escala de los datos
Arbol de Decisión
Modelo basado en reglas if-then
Divide los datos según características relevantes
Fácil de interpretar y visualizar
Riesgo de sobreajuste
Utilizado en sistemas de decisión y clasificación empresarial
3. Aprendizaje No Supervisado
Concepto
No existen etiquetas en los datos
El modelo descubre patrones ocultos
Clustering
K-Means
Algoritmo de clustering más utilizado
Define un número k de grupos
Asigna datos al centroide más cercano
Recalcula centroides iterativamente
Requiere definir k previamente
Aplicaciones
Segmentación de clientes
Análisis de comportamiento
Compresión de datos
4. Aprendizaje Profundo (Deep Learning)
Concepto
Subcampo del Machine Learning
Basado en redes neuronales profundas
Capaz de aprender representaciones complejas
Redes Neuronales
Inspiradas en el cerebro humano
Compuestas por capas
Capa de entrada
Capas ocultas
Capa de salida
Visión Artificial
Object Detection (Detección de Objetos)
Identifica y localiza objetos dentro de una imagen
Combina clasificación y localización
Produce bounding boxes
Modelos comunes
YOLO
SSD
Faster R-CNN
Aplicaciones
Vehículos autónomos
Seguridad y vigilancia
Análisis médico

ACTIVIDAD 01

Para esta actividad, nos basaremos en ejercicios del website **http://www.aprendemachinelearning.com/.**
Utilizaremos su material disponible en el repositorio de GitHub https://github.com/jbagnato/machine-learning (Fork del repositorio también disponible en https://github.com/sergarb1/machine-learning).

La actividad consiste en:

- Descarga localmente el repositorio.
- instala (si es necesario) las bibliotecas de Jupyter ("pip3 install jupyter") los plugins de Jupyter Notebooks para Visual Studio Code
  - Documentación Jupyter Notebooks **https://jupyter-notebook.readthedocs.io/en/latest/**
  - Notebooks Jupyter Plugin: **https://marketplace.visualstudio.com/items?itemName=ms-toolsai.jupyter**
  - Tutorial básico Jupyter Notebooks:
    - https://code.visualstudio.com/docs/datascience/jupyter-notebooks
    - https://code.visualstudio.com/learn/educators/notebooks

- Explora, pone en marcha y comprenda en profundidad los siguientes "Jupyter Notebooks" del repositorio:

  - Obligatorios:
    - “Manipulacion_datos_pandas.ipynb”
    - “Ejercicio_Regresion_Lineal.ipynb”.

  - A elegir 2 de:
    - “Ejercicio_k-NearestNeighbor.ipynb”.
    - “Ejercicio_K_Means.ipynb”
    - “Ejercicio_Arbol_de_Decision.ipynb”
    - “Ejercicio_Object_Detection.ipynb”

- Presenta 4 vídeos (o si no puede ser, concierta cita para hacerlo en directo) donde explicas en detalle los 4 "Jupyter Notebooks". Puedes añadir comentarios al código. Tendrás que explicar:
  - Líneas de código.
  - Función.
  - Resultados (cuando haya).

- Importante: los vídeos serán privados para mí con el único fin de fe la corrección. En los vídeos sólo se usarán como forma de evaluar lo que sabéis. No evaluaré la calidad del vídeo (no es necesario que haga una edición perfecta, ni que el vídeo esté perfecto, puede haber cortes o momentos de silencio, puede hacerse en castellano, etc.).

Machine Learning

Machine Learning (Aprendizaje Automático) es una rama de la Inteligencia Artificial que se centra en desarrollar algoritmos y modelos que permiten a las computadoras aprender de los datos. A diferencia de la programación tradicional, donde las instrucciones son explícitas, en Machine Learning se alimentan datos y resultados, y el sistema construye reglas por sí mismo para hacer predicciones o tomar decisiones.

En el vasto mundo del Machine Learning, los métodos se dividen en tres categorías principales, cada una con su propio enfoque y conjunto de aplicaciones.

Aprendizaje Supervisado

Es un paradigma de Machine Learning en el que un modelo es entrenado utilizando un conjunto de datos etiquetado, es decir, datos de entrada con respuestas correspondientes conocidos. El objetivo de este enfoque es que el modelo adquiera la capacidad de predecir resultados sobre datos no vistos, minimizando el error de predicción con respecto a las etiquetas reales. Este enfoque es esencial en aplicaciones donde se requiere un alto grado de precisión y se dispone de un conjunto de datos confiable para el entrenamiento.

Aprendizaje No Supervisado

Se caracteriza por la ausencia de etiquetas en los datos. En este paradigma, el modelo examina el conjunto de datos para identificar patrones, estructuras o agrupamientos subyacentes sin intervención humana. Este tipo de aprendizaje es particularmente útil en situaciones donde se busca explorar la naturaleza de los datos y descubrir relaciones inherentes, facilitando análisis descriptivos y generando insights significativos a partir de conjuntos de datos complejos.

Aprendizaje por Refuerzo

El aprendizaje por refuerzo es un enfoque en el que un agente toma decisiones en un entorno dinámico, aprendiendo a través de la retroalimentación que recibe en forma de recompensas o penalizaciones. Este modelo interactúa de manera continua con su entorno, evaluando las consecuencias de sus acciones para optimizar su rendimiento a largo plazo. El aprendizaje por refuerzo se aplica en contextos donde la toma de decisiones simultánea y adaptativa es crucial, permitiendo a los modelos adaptarse y mejorar en función de experiencia acumulada.

Cada tipo de aprendizaje en Machine Learning tiene su propio propósito y metodología, adaptándose a distintas necesidades y desafíos. Estos métodos son fundamentales para desarrollar modelos que pueden resolver problemas complejos y ofrecer soluciones innovadoras.

Tipos de Modelos

Modelo de Regresión Lineal

Es una técnica de aprendizaje supervisado utilizada para predecir un valor numérico continuo. Se basa en identificar una relación lineal entre una variable dependiente y una o más variables independientes. Por ejemplo, se puede usar para prever el precio de una vivienda en función de factores como el tamaño, la ubicación y el número de habitaciones. A través del análisis de los datos históricos, el modelo establece una ecuación que permite hacer predicciones precisas.

k-Nearest Neighbor (k-NN)

Se emplea en la clasificación supervisada y funciona clasificando un dato según a qué grupo pertenecen sus "vecinos" más cercanos. Cuando se presenta un nuevo dato, el modelo determina su clase al observar las etiquetas de los "k" ejemplos más cercanos en el espacio de características.

Árbol de Decisión

Este modelo que utiliza decisiones lógicas en forma de un diagrama en cascada para clasificar datos. Comienza con una pregunta sobre una característica de los datos, dividiendo el conjunto en ramas de acuerdo con las respuestas. Cada hoja del árbol representa una decisión final, lo que lo hace fácil de interpretar.

Object Detection (Detección de Objetos)

Se ocupa de identificar y clasificar objetos dentro de una imagen. Este modelo, también basado en técnicas de aprendizaje supervisado, no solo determina qué objetos están presentes, sino que también determina sus ubicaciones exactas en la imagen, utilizando cajas delimitadoras. Esto es esencial en aplicaciones como la conducción autónoma y la seguridad.

K-Means

Se centra en el aprendizaje no supervisado y se utiliza para el agrupamiento de datos. Su objetivo es dividir un conjunto de datos en "K" grupos (clusters) sin ninguna etiqueta previa. El modelo asigna cada dato al grupo más cercano según una medida de distancia, ajustando continuamente los centros de los grupos para minimizar la variabilidad dentro de cada cluster. K-Means es ideal para descubrir patrones en grandes conjuntos de datos, como en el análisis de segmentación de clientes.

Estos modelos son herramientas fundamentales en el campo del aprendizaje automático y cada uno tiene su propio conjunto de aplicaciones y ventajas, permitiendo resolver una variedad de problemas de manera efectiva.

PREPARACION DEL ENTORNO

Descarga del Repositorio

Abrir la terminal o consola de comandos y ejecutar:

git clone https://github.com/jbagnato/machine-learning.git

cd machine-learning

git status

```
miriam@                              $ git clone --depth 1 https://github.com/jbagnato/machine-learning.git
Clonando en 'machine-learning'...
remote: Enumerating objects: 74, done.
remote: Counting objects: 100% (74/74), done.
remote: Compressing objects: 100% (72/72), done.
remote: Total 74 (delta 0), reused 63 (delta 0), pack-reused 0 (from 0)
Recibiendo objetos: 100% (74/74), 46.84 MiB | 4.71 MiB/s, listo.
miriam@                              $ cd machine-learning
git status
En la rama master
Tu rama está actualizada con 'origin/master'.

nada para hacer commit, el árbol de trabajo está limpio
miriam@                              :~/machine-learning$
```

Instalación de Dependencias

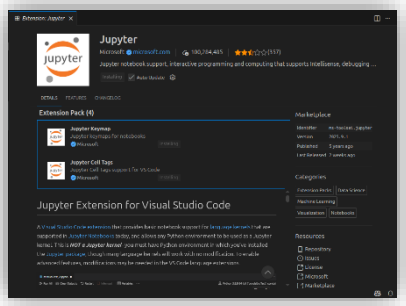
- Tener Python instalado.
- Crea el entorno virtual llamado 'env' → ***python3 -m venv env***
- Activar el entorno → ***source env/bin/actívate*** (el prompt de lterminal cambia y muestra (env) al principio.)
- Instalar Jupyter y las bibliotecas básicas requeridas:

  pip install jupyter pandas numpy matplotlib seaborn scikit-learn

```
miriam@                              ~/machine-learning$ python3 -m venv env
miriam@                              ~/machine-learning$ source env/bin/activate
(env) miriam@                        :~/machine-learning$
```

Configuración de Visual Studio Code

Abrir VS Code, ira vista de Extensiones (Ctrl+Shift+X) y buscar e instalar "Jupyter" de Microsoft.
Al abrir un archivo .ipynb, VS Code pedirá seleccionar un "Kernel" (el intérprete de Python); seleccionar el que se tiene instalado.

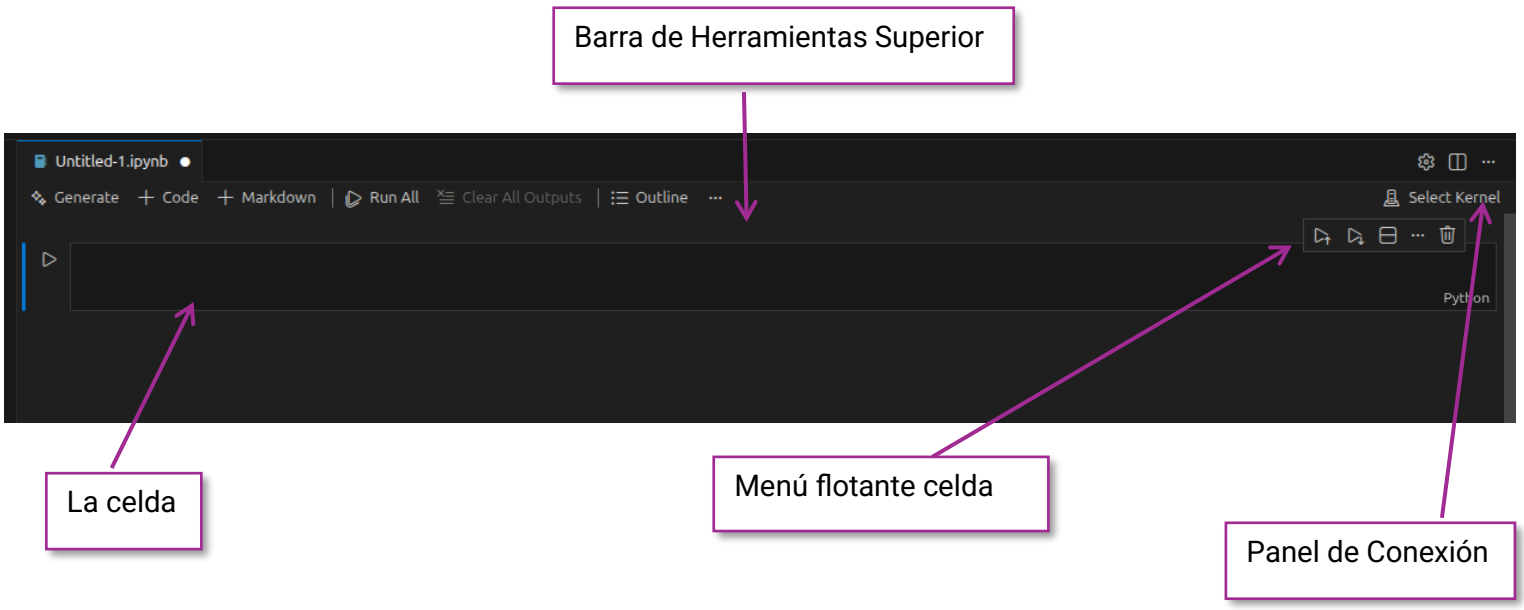


Jupyter (Juypter Notebooks)

Es una aplicación de código abierto que permite crear y compartir documentos que contienen código en vivo, ecuaciones, visualizaciones y texto narrativo. Es ampliamente utilizado en la ciencia de datos, machine learning, y otras disciplinas que requieren análisis avanzado o investigación.

Para crear un notebook:

- Presiona las teclas Ctrl + Shift + P para abrir la Paleta de Comandos.
- Escribe: Create: New Jupyter Notebook y pulsa Enter.



1. **Barra de Herramientas Superior**

   Esta barra controla el notebook completo.

   - + Code: Crea un nuevo "rectángulo" (celda) para escribir código Python.
   - + Markdown: Crea una celda de texto. Sirve para poner títulos , no para código.
   - Run All: Ejecuta todas las celdas del archivo en orden, desde la primera hasta la última. Es muy útil para verificar que todo el proyecto funciona de un tirón.
   - Clear All Outputs: Borra los resultados (gráficos, tablas, textos) que aparecieron debajo de las celdas después de ejecutarlas. Deja el notebook "limpio".
   - Outline: Abre un índice a la izquierda que permite saltar rápido entre las secciones del código si se usa títulos.
   - Generate: Asistencia de Inteligencia Artificial integrada en Visual Studio Code.

2. El Panel de Conexión

- Select Kernel: qué versión de Python usar para procesar el código y elegir el entorno virtual (env) .
- Engranaje (Settings): Abre la configuración específica del Notebook y de la extensión de Jupyter, sirve para cambiar temas visuales o comportamientos de cómo se muestran los resultados.
- Ventana/Libro (Variables): Abre el Explorador de Variables.

3. La Celda

Las celdas son las unidades básicas de un Notebook en Jupyter. Existen varios tipos de celdas:

- Celdas de Código: Permiten escribir y ejecutar código. El resultado se muestra directamente debajo de la celda.
- Celdas de Markdown: Permiten crear texto formateado usando un lenguaje de marcado. Sirven para documentar el código y explicar los resultados, mediante encabezados, listas, enlaces, etc.
- Celdas de Ecuaciones: Utilizan LaTeX para incluir fórmulas matemáticas dentro del Notebook.

- El botón de "Play" (triángulo a la izquierda): Ejecuta solo esa celda.
- El área oscura: Es donde escribes el código.
- Python (abajo a la derecha de la celda): confirma que el lenguaje que estás usando en esa celda.

4. Menú Flotante de la Celda

Cuando pasas el ratón sobre una celda, aparecen estos iconos:

- Flechas arriba/abajo: Sirven para mover esa celda de lugar en el documento.
- Icono de papelera (Trash): Borra esa celda específica.
- Tres puntos (...): Abre más opciones:
  - ✓ Split Cell (Dividir celda): Corta una celda larga en dos más pequeñas.
  - ✓ Merge with Next Cell (Unir): Junta la celda actual con la de abajo.
  - ✓ Copy/Paste Cell: Para duplicar código rápidamente.
  - ✓ Etc.

5. Salida de Datos

Cuando se ejecuta una celda de código, cualquier salida (como gráficos, tablas o texto) se muestra justo debajo, permitiendo una interacción inmediata y feedback sobre el trabajo realizado.

Bibliotecas de Python para Análisis de Datos y Machine Learning

Pandas

Pandas es una biblioteca de Python que proporciona estructuras de datos y herramientas de análisis de datos. Es fundamental para el manejo y manipulación de datos en formato tabular (DataFrames).

Uso Principal: Limpieza, manipulación, análisis y visualización de datos.

Funciones Clave:

- pd.read_csv(): Cargar datos desde archivos CSV.
- df.head(), df.tail(): Visualizar las primeras o últimas filas.
- df.groupby(): Agrupar datos por una o más columnas.

NumPy

NumPy es una biblioteca que proporciona soporte para arreglos multidimensionales y funciones matemáticas de alto rendimiento.

Uso Principal: Cálculos numéricos y operaciones con arreglos.

Funciones Clave:

- np.array(): Crear arreglos Numpy.
- np.mean(), np.std(): Calcular media y desviación estándar.
- Operaciones matemáticas y lógicas sobre arreglos.

Matplotlib

Matplotlib es una biblioteca de visualización que permite crear gráficos en Python. Es extremadamente versátil y se puede utilizar para crear una amplia variedad de visualizaciones.

Uso Principal: Visualización de datos.

Funciones Clave:

- plt.plot(): Crear gráficos de líneas.
- plt.scatter(): Crear gráficos de dispersión.
- plt.show(): Mostrar la figura creada.

Seaborn

Seaborn es una biblioteca basada en Matplotlib que proporciona una interfaz más sencilla para la creación de gráficos estadísticos. Se enfoca en la visualización de datos con el uso de temáticas.

Uso Principal: Visualización estadística de datos de manera más atractiva y fácil.

Funciones Clave:

sns.scatterplot(): Crear gráficos de dispersión con estilos mejorados.
sns.boxplot(): Crear diagramas de caja para visualizar la distribución.
sns.heatmap(): Crear mapas de calor para observar correlaciones.

Scikit-learn

Scikit-learn es una biblioteca muy utilizada para la implementación de algoritmos de machine learning y técnicas de análisis predictivo.

Uso Principal: Modelado, evaluación, y predicciones en machine learning.

Funciones Clave:

train_test_split(): Dividir un conjunto de datos en entrenamiento y prueba.
LinearRegression(), KNeighborsClassifier(): Implementaciones de algoritmos de machine learning.
cross_val_score(): Evaluar el rendimiento del modelo mediante validación cruzada.

Estas bibliotecas son esenciales para el análisis de datos y el desarrollo de modelos de machine learning en Python. , que junto con Jupyter crean un ecoistema interactivo que permite a los usuarios escribir y ejecutar código, visualizar resultados y documentar procesos de manera fluida.

Análisis de Notebooks

Notebook	Propósito	Concepto Clave
Pandas	Limpieza y análisis de datos.	DataFrames, Series, filtrado. Indexación, agrupación, manejo de valores nulos, operaciones combinadas.
Regresión Lineal	Predecir un valor numérico continuo.	Recta de regresión, error cuadrático. Suposiciones (linealidad, homocedasticidad), coeficientes, R^2 (coeficiente de determinación).
k-NN	Clasificar puntos según sus vecinos.	Distancia euclidiana, votos de clases. Escalamiento de características, elección de k, complejidad computacional.
Árbol de Decisión	Clasificar mediante reglas lógicas.	Nodos, entropía, ganancia de información. Sobreajuste, poda del árbol, criterios de división (Gini, entropía).

Casi todos tus notebooks siguen este orden:

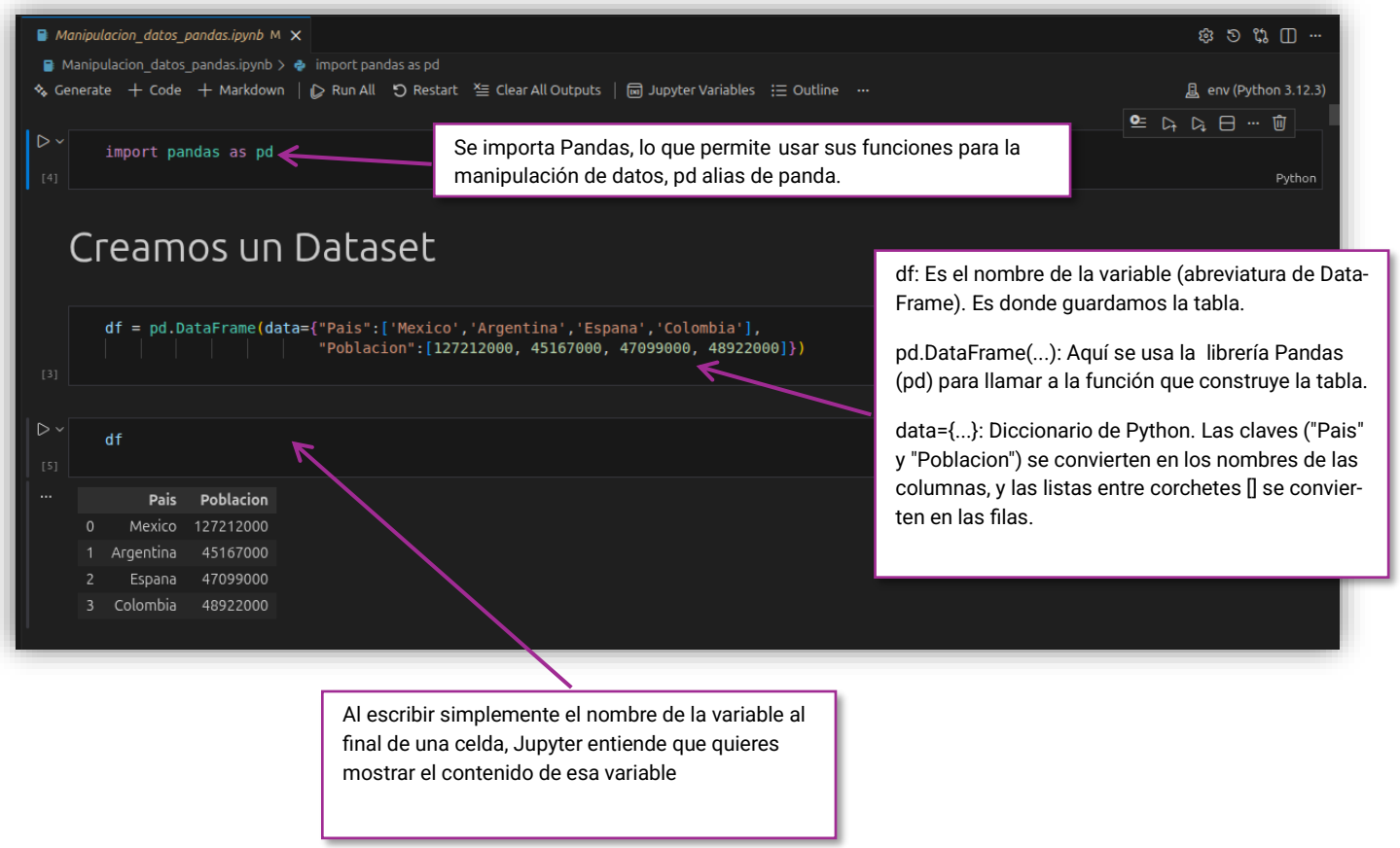
1. **Cargar datos:** Abrir el archivo CSV.
2. **Limpiar:** Quitar datos que faltan o están mal.
3. **Dividir:** Separar los datos en "Entrenamiento" (para que aprenda) y "Test" (para ponerle un examen y ver si aprendió bien).
4. **Entrenar (fit):** Es el momento donde la computadora estudia.
5. **Predecir (predict):** Usar el modelo con datos nuevos.

1. Manipulación de Datos con Pandas (Manipulacion_datos_pandas.ipynb)

https://www.aprendemachinelearning.com/analisis-exploratorio-de-datos-pandas-python/

Este Notebook se centra en la manipulación de datos utilizando la biblioteca **Pandas** de Python. Su función es cargar, limpiar y organizar datos antes de usarlos en IA.

El objeto principal de Pandas es el DataFrame, que es como una tabla de Excel altamente optimizada.



Los resultados iniciales brindan una visión general de los datos, lo que permite identificar los tipos de variables y la cantidad de datos disponibles. Esto es crucial para decidir qué tipo de análisis o modelado se puede realizar posteriormente.

Algunas de la funciones importantes en Pandas:

a. Carga y Visualización Inicial

Estas funciones sirven para meter los datos al programa y ver qué contienen.

- pd.read_csv('archivo.csv'): Carga un archivo CSV y lo convierte en un DataFrame (una tabla).

- df.head(n): Muestra las primeras *n* filas (por defecto 5). Es la mejor forma de verificar que los datos se cargaron bien.
- df.tail(): Al contrario que head, muestra las últimas filas del conjunto de datos.
- df.sample(n): Elige *n* filas al azar para que veas una muestra variada de los datos.

b. Inspección y Estadísticas

- df.info(): Te dice cuántas filas hay, el nombre de las columnas y si falta información (valores nulos).
- df.describe(): Genera un resumen estadístico: media, desviación estándar, valores mínimos y máximos.
- df.shape: devuelve una tupla con el número de (filas, columnas).
- df.columns: Lista todos los nombres de las columnas disponibles.

c. Selección y Filtrado

Sirve para "recortar" la tabla y quedarse solo con lo que interesa.

- df['columna']: Selecciona una sola columna.
- df[['col1', 'col2']]: Selecciona varias columnas a la vez.
- df.iloc[fila, columna]: Selecciona datos usando su posición numérica (como en una matriz).
- df[df['edad'] > 18]: Filtra la tabla basándose en una condición lógica.

d. Limpieza de Datos (Muy importante para IA)

Los modelos de Machine Learning no suelen aceptar celdas vacías.

- df.isnull().sum(): Cuenta cuántos valores vacíos hay en cada columna.
- df.dropna(): Elimina todas las filas que tengan al menos un valor vacío.
- df.fillna(valor): Rellena los huecos vacíos con un valor específico (como el promedio o un cero).
- df.drop_duplicates(): Borra las filas que estén repetidas.

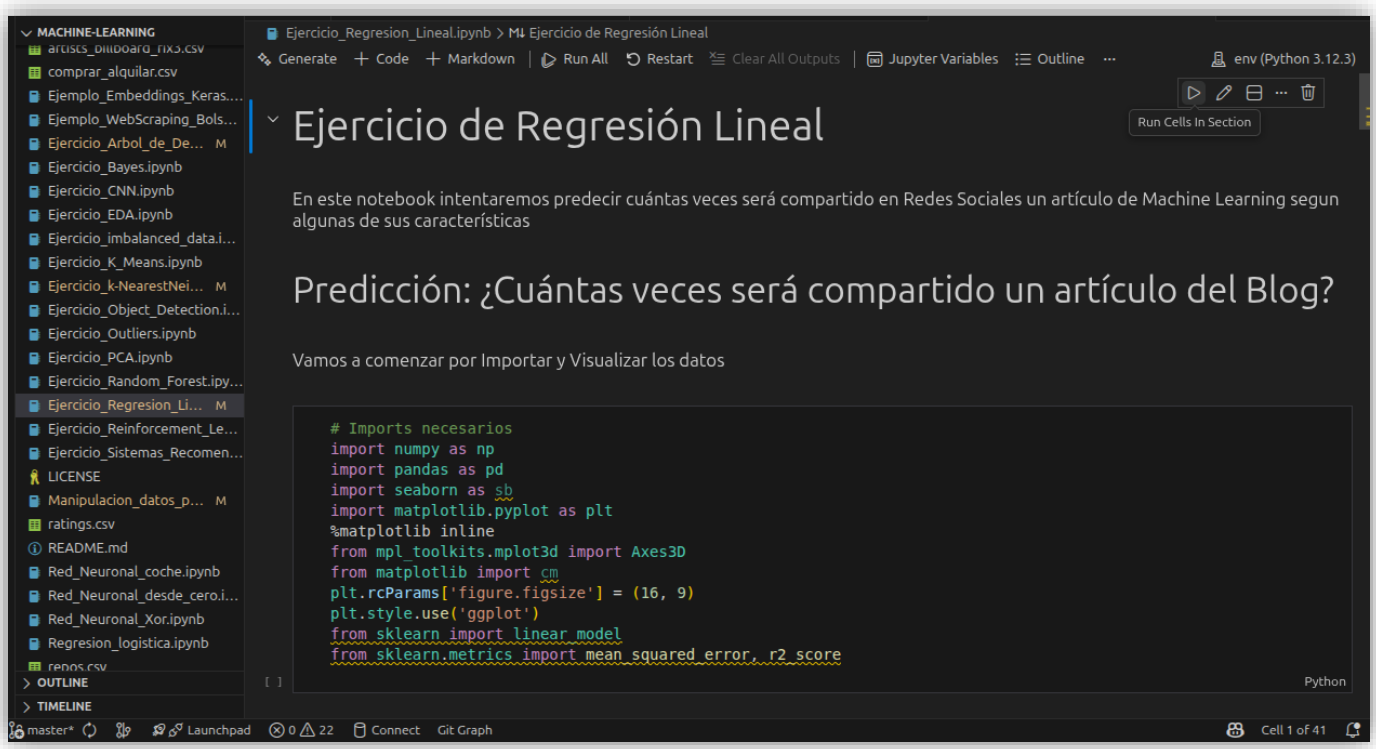
e. Manipulación y Agrupación

- df.groupby('columna').mean(): Agrupa los datos por una categoría y calcula el promedio (ej: promedio de ventas por ciudad)
- df.sort_values(by='columna'): Ordena la tabla de forma ascendente o descendente-
- df.rename(columns={'viejo': 'nuevo'}): Cambia el nombre de las columnas para que sean más fáciles de entender.

2. Regresión Lineal (Ejercicio_Regresion_Lineal.ipynb)

https://www.aprendemachinelearning.com/regresion-lineal-en-espanol-con-python/

Este Notebook introduce la técnica de regresión lineal. Es esencial para entender la relación entre variables en un conjunto de datos .



La regresión lineal es un método estadístico que trata de modelar la relación entre una variable dependiente (*Y*) y una o más variables independientes (*X*).

El objetivo es encontrar una línea recta que mejor se ajuste a los puntos de datos dispersos en un gráfico. Esta línea nos permite realizar **predicciones** sobre valores que aún no conocemos.

a. La Ecuación de la Recta

El algoritmo intenta resolver una ecuación matemática básica: $Y = mX + b$

- *Y*: Es el resultado que queremos predecir (ej. cuántas veces se compartirá el artículo).
- *X*: Es el dato de entrada (ej. cantidad de palabras).
- *m* (Pendiente): Determina qué tan inclinada está la recta. En programación se le llama coeficiente.
- *b* (Intersección): Es el punto donde la recta cruza el eje vertical cuando *X* es cero.

b. El modelo utiliza un proceso llamado *Mínimos Cuadrados Ordinarios*.

Traza una línea inicial: Empieza con una posición aleatoria.

Mide el error: Calcula la distancia vertical entre cada punto real y la línea. Esa distancia es el "error" o residuo.

Suma los errores: Eleva esos errores al cuadrado (para que siempre sean positivos) y los suma. A esto se le llama Función de Coste.

Ajusta la línea: El algoritmo ajusta los valores de *m* y *b* repetidamente hasta que esa suma de errores sea la mínima posible.

c. Tipos de Regresión Lineal

- **Simple**: Cuando solo usas una variable para predecir (ej. "Palabras" para predecir "Shares").
- **Múltiple**: Cuando usas varias variables al mismo tiempo (ej. "Palabras" + "Imágenes" + "Día de la semana" para predecir "Shares"). En este caso, en lugar de una línea, el modelo busca un plano o un hiperplano.

d. Definir las variables predictoras (X) y la variable objetivo (y)

X = df[['feature1', 'feature2']]

y = df['target']

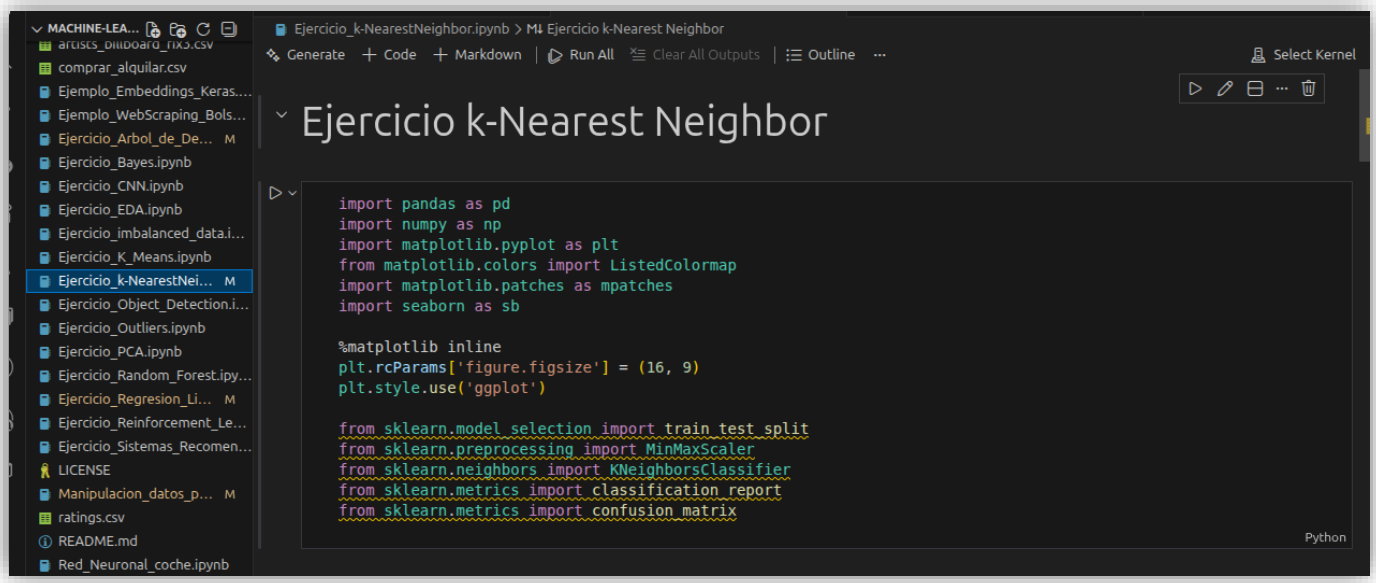
Creación y ajuste del modelo → *model = LinearRegression( )*

Entrenamiento → model.fit(X, y)

Después de entrenar el modelo, se puede evaluar usando diferentes métricas como R^2 o error cuadrático medio. Estos resultados indican qué tan bien el modelo se ajusta a los da tos y su capacidad para predecir la variable objetivo.

3. k-Nearest Neighbor (Ejercicio_k-NearestNeighbor.ipynb)

https://www.aprendemachinelearning.com/clasificar-con-k-nearest-neighbor-ejemplo-en-python/



El algoritmo k-Nearest Neighbor (k-NN) es utilizado para clasificar datos en función de las distancias a los vecinos más cercanos.

Las variables predictoras y la etiqueta se definen para el modelo:

```
X = df[['feature1', 'feature2']]

y = df['label']
```

X : Son las variables que describen los datos. En un problema de clasificación, cada observación se representa a través de un conjunto de características.

Etiquetas (y): Son las clases o categorías que se quieren predecir. Por ejemplo, en una clasificación de reseñas, las etiquetas podrían ser diferentes niveles de satisfacción, como 1 estrella, 2 estrellas, etc.

Se define *k* = 7. Esto significa que para clasificar un punto nuevo, el algoritmo buscará a los 7 vecinos más cercanos en el espacio de características y le asignará la clase mayoritaria.

knn.fit(X_train, y_train): El modelo "aprende" la posición de los datos de entrenamiento.

El código genera un mapa de colores (mesh grid) que muestra las zonas de influencia de cada categoría.

- Zonas de color: Representan la predicción del modelo en cualquier punto del plano.

- Puntos: Son las observaciones reales. Esto ayuda a visualizar qué tan "limpias" o "ruidosas" son las fronteras entre sentimientos.

Los resultados mostrarán las etiquetas predichas y permitirá calcular métricas de precisión y recuperación. La comparación entre etiquetas reales y predichas ayuda a evaluar el rendimiento del modelo.

predict(): Devuelve la clase final (ej. 1 estrella).

predict_proba(): Devuelve la probabilidad de pertenecer a cada una de las 5 clases (ej. 80% de probabilidad de ser 1 estrella, 20% de ser 2 estrellas).

4. Árbol de decisión (Ejercicio_Arbol_de_Decision.ipynb)

https://www.aprendemachinelearning.com/arbol-de-decision-en-python-clasificacion-y-prediccion/

Un árbol de decisión actúa como un diagrama de flujo donde cada nodo representa una pregunta sobre las características de los datos. A partir de las respuestas, el árbol se divide, creando caminos que llevan a decisiones o clasificaciones finales.



Estructura del Árbol

- Nodos: Cada nodo representa una característica de los datos y una pregunta que se realiza para dividir los datos.
- Ramas: Las conexiones entre nodos representan las respuestas a las preguntas. Cada rama refleja una posible decisión en función de la respuesta obtenida.
- Hojas: Los nodos finales del árbol, donde se asigna una clase o resultado.

En cada nodo, se selecciona la pregunta que mejor separa los datos de acuerdo con algún criterio. Comúnmente se utilizan métricas como Gini Impurity o Entropía. Estas métricas evalúan cuán "mezcladas" están las clases en un conjunto de datos.

Basado en la pregunta seleccionada, los datos se dividen en dos o más grupos. Este proceso se repite para cada subgrupo hasta que se cumpla un criterio de parada (por ejemplo, profundidad máxima, número mínimo de muestras en un nodo).

La profundidad del árbol se refiere al número de nodos desde la raíz hasta las hojas. Un árbol muy profundo puede overfitear los datos, es decir, ajustarse demasiado a los datos de entrenamiento y perder capacidad de generalización a nuevos datos.

Para clasificar un nuevo dato, el algoritmo comienza en la raíz del árbol y sigue las ramas basándose en las respuestas a las preguntas planteadas en cada nodo hasta llegar a una hoja. La clase en la hoja determina la predicción final.

- Las métricas utilizadas para evaluar el desempeño del árbol de decisión incluyen:
- Precisión: Porcentaje de clasificaciones correctas respecto al total de datos.
- Recall: Proporción de instancias relevantes que el modelo ha conseguido identificar correctamente.
- F1 Score: Combina precisión y recall en una sola métrica.

BIBLIOGRAFIA

- Apuntes del Profesor.
- Python para todos. Explorando la información con Python 3. Charles R. Severance
- https://docs.python.org/es/3/library/copy.html
- https://markmap.js.org/
- https://developers.google.com/machine-learning/intro-to-ml?hl=es-419
- https://www.youtube.com/watch?v=gfBQN1ndxdA
- https://www.youtube.com/watch?v=kN5fZldDBEc
- https://www.youtube.com/watch?v=FlsoBiteuPM
- https://www.markdownguide.org/
- https://dillinger.io/
- https://joedicastro.com/pages/markdown.html
- https://www.youtube.com/watch?v=77Ggk1uzO2A
- https://www.youtube.com/watch?v=RDGfF20kdFQ
- https://machinelearningparatodos.com/arboles-de-decision-en-python/
- https://www.youtube.com/watch?v=PZqkabiRuBA
- https://www.youtube.com/watch?v=XNWKJsDNScw